



Introduction to OpenCog Hyperon



Tollan Sitotaw
July 26, 2024



The Open Cognition Project (OpenCog)

OpenCog is an open source framework for Artificial General Intelligence. **How?**

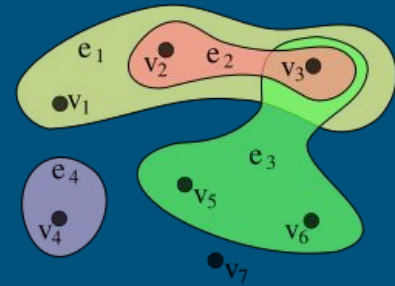
It inherited a large body of software code and software designs from Novamente LLC developed during the period 2001-2007.

The OpenCog project itself was founded in 2008.

It has two primary components:

- OpenCog AtomSpace
- OpenCog Atomese language

OpenCog Atomspace



The **OpenCog AtomSpace** is a knowledge representation (KR) database and the associated query/reasoning engine to fetch and manipulate that data, and perform reasoning on it.

It is a kind of graph database, but with the ability to connect nodes with any number of links and connect any number of nodes with links (**Hypergraph**).

It is a collection of Atomic representations of knowledge (**Atoms**) which could either be Nodes and Links.

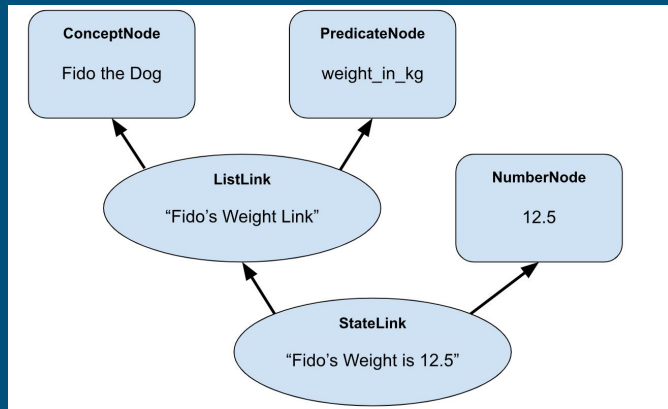
OpenCog Atomese Language

Atomese is the concept of writing programs with atoms.

Atomese is a graph language, optimized for manipulation by algorithms.

- Here's how to express *"Fido the Dog's weight in kg is 12.5"* in Atomese:

```
(StateLink
  (ListLink
    (Concept "Fido the Dog")
    (Predicate "weight_in_kg")
  )
  (NumberNode 12.5)
)
```



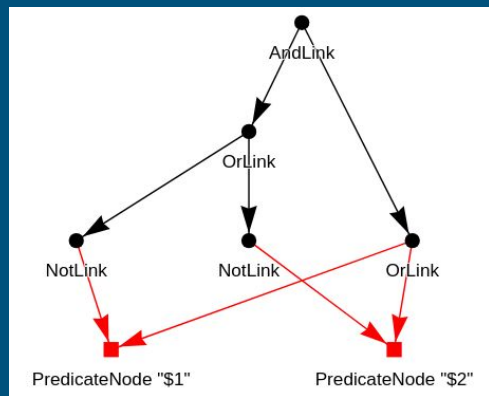
OpenCog MOSES

MOSES is a different approach to evolutionary programming.

MOSES performs supervised learning, and thus requires either a scoring function or training data to be specified as input.

As output, it generates a program that, approximates the scoring function.

\$1	\$2	RESULT
0	0	0
1	1	0
1	0	1
0	1	1



OpenCog MOSES

The **MOSES algorithm** starts from an empty program (or an exemplar chosen from a metapopulation) and constructs a representation by adjusting the knobs of the program.

The best-possible knob settings for the mutated population of programs derived from a representation (deme) are found by using well-known optimization algorithms.

All program evolution algorithms tend to produce redundant programs. To avoid this, MOSES performs reduction at each stage to bring the program into normal form.

The above two techniques, optimization and normalization, allow MOSES to outperform standard genetic programming systems.

OpenCog Pattern Miner

The **Pattern Miner** is a system for discovering frequent and **interesting** patterns in the AtomSpace.

Most of frequent patterns are boring. We use a surprisingness measure to find interesting patterns.

It can be used to find pattern matcher queries that would return frequent and interesting collection of matches.

Use Cases:

- Mining of dialogue histories, for learning dialogue patterns for a person.
- Mining of sets of genomic datasets or medical patient records, to find surprisingly common combinations of features.

Probabilistic Logic Network (PLN)

PLN is a novel conceptual, mathematical and computational approach to uncertain inference.

PLN uses probabilities in place of crisp (true/false) truth values, and fractional uncertainty in place of crisp known/unknown values.

The current version of PLN has been used in narrow-AI applications such as the inference of biological hypotheses from knowledge extracted from biological texts via language processing.

OpenCog Hyperon

OpenCog Hyperon is a substantially revised, novel version of OpenCog.

Same as OpenCog, it has two main components:

- **Hyperon Atomspace**: A more scalable, flexible and distributed Atomspace representation.
- **Hyperon Atomese**: A successor to the OpenCog Classic Atomese language with clear semantics supporting meta-language features, different types of inference, etc.

The Hyperon Atomese language is called **MeTTa** (Meta Type Talk).

MeTTa

MeTTa is designed as a language of the cognitive architecture OpenCog Hyperon focused on grounded knowledge representation and reasoning with inference control and neural-symbolic integration in mind.

The basic atomic elements of the MeTTa language are symbols, which are represented by sequences of arbitrary Unicode characters.

```
(Hi is a symbol)
(____ is a symbol)
(@&#<> is a symbol)
```

- Learn more about the MeTTa language in the [MeTTa tutorial](#).

MeTTa Atomspace

A MeTTa program can be represented as a subgraph of an Atomspace.

```
(gene ENSG0001)
(gene name (gene ENSG0001) LINC01409)
(gene type (gene ENSG0001) lncRNA)
(chr (gene ENSG0001) chr1)

(gene ENSG0002)
(gene name (gene ENSG0002) FGR)
(gene type (gene ENSG0002) protein_coding)
(chr (gene ENSG0002) chr2)
```

```
(transcribed_to (gene ENSG0001)
(transcript ENST0011))

(transcribed to (gene ENSG0002)
(transcript ENST0022))
```

```
(transcript ENST0011)
(transcript name (transcript ENST0011) LINC01409-20)
(transcript type (transcript ENST0011) lncRNA)
(chr (transcript ENST0011) chr1)
(gene_name (transcript ENST0011) LINC01409)

(transcript ENST0022)
(transcript name (transcript ENST0022) FGR-204)
(transcript_type (transcript ENST0022) retained_intron)
(chr (transcript ENST0022) chr2)
(gene_name (transcript ENST0022) FGR)
```

MeTTa Atomspace

A MeTTa program can also be used to query and rewrite portions of an Atomspace.

```
; 'gene' node with properties
(gene ENSG0001)
(gene_name (gene ENSG0001) LINC01409)
(gene_type (gene ENSG0001) lncRNA)
(chr (gene ENSG0001) chr1)

; 'transcript' node with properties
(transcript ENST0011)
(transcript_name (transcript ENST0011) LINC01409-20)
(transcript_type (transcript ENST0011) lncRNA)
(chr (transcript ENST0011) chr1)

; 'transcribed_to' link between 'gene' and 'transcript'
(transcribed_to (gene ENSG0001) (transcript ENST0011))
```

The 'match' keyword

You can use the match function to query knowledge from a MeTTa Atomspace.

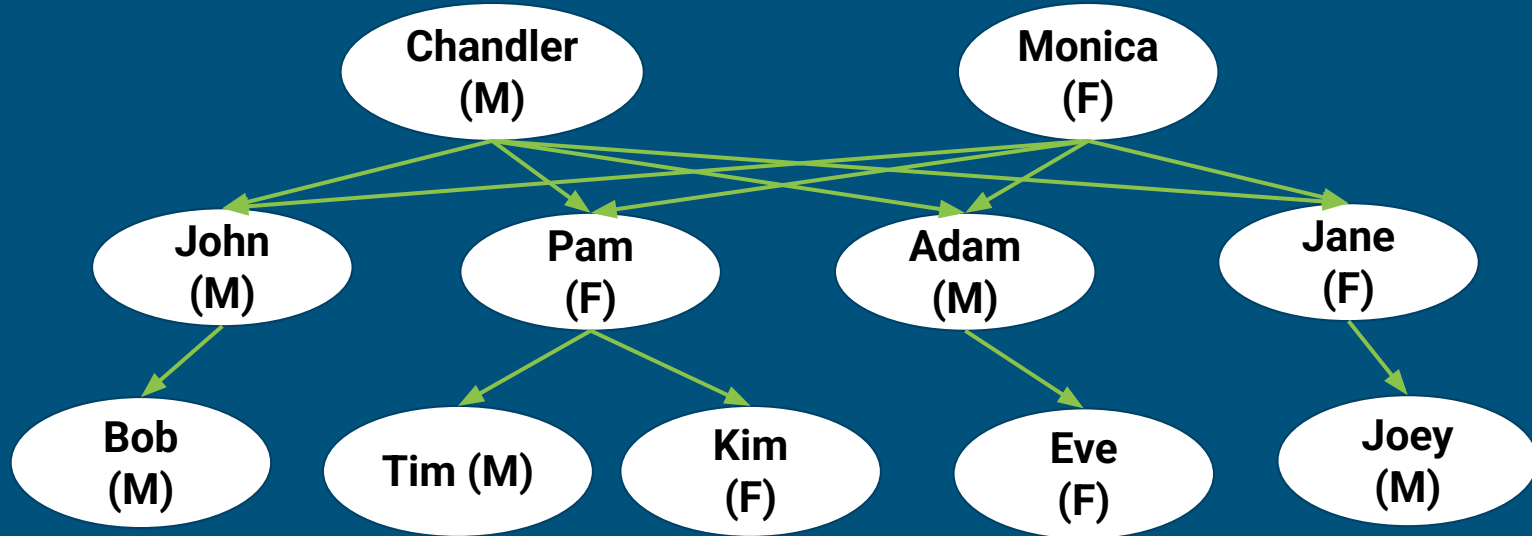
- **'match'** takes three arguments: *search_space*, *pattern(s)_to_match* & *output_expression*

```
!(match &self (transcribed_to (gene $gene) (transcript $transcript))
  ($gene is_transcribed_to $transcript)
)
; [(ENSG0001 is_transcribed_to ENST0011)]
```

```
!(match &self
  (,
    (transcribed to (gene $gene) (transcript $transcript))
    (gene_name (gene $gene) $gene_name)
    (transcript_name (transcript $transcript) $transcript_name)
  )
  ($gene_name is_transcribed_to $transcript_name)
)
; [(LINC01409 is_transcribed_to LINC01409-20)]
```

Exercise 1

- Represent this family tree in an Atomspace:



Exercise 2

Implement the following match queries for the family-tree Atomspace.

- Get the **Parents** of John.
- Get the **Mother** of Pam.
- Get the **Siblings** of Pam.
- Get the **Uncles** of Tim.
- Get the **Nephews** of John.
- Get all **Predecessors** of Bob.

Functional Programming in MeTTa

Characteristics of functional programming:

- Immutable state
- Recursion instead of loops
- Pattern matching instead of conditionals
- Functions are pure (have no side-effects)
- Functions are first class citizens

Characteristics of MeTTa:

- All above characteristics
- Non-determinism

Types & Metatypes in MeTTa

Atom is a fundamental building block of all the data in an Atomspace.

There are four basic types of Atoms (*metatypes*) in MeTTa:

- **Symbol** - Any arbitrary sequence of unicode characters.
- **Expression** - Atoms which can encapsulate other atoms including other expressions.
- **Grounded** - Represents sub-symbolic data in the Atomspace. Grounded Atoms could have a String, Number or Bool type.
- **Variable** - Symbols used to create patterns which can bind to any Atom.

Defining Functions in MeTTa

- We use the `'=`' symbol to define a function in MeTTa.
- The `'=`' symbol takes two parameters: the pattern to be matched and the output to be returned.

```
(= (h) (Hello world))  
! (h) ; (Hello world)  
  
(= (only-a A) (Input A is accepted))  
! (only-a A) ; (Input A is accepted)  
! (only-a B) ; (only-a B)  
  
(= (swap (Pair $x $y)) (Pair $y $x))  
! (swap (Pair A B)) ; (Pair B A)
```

Conditional Statements in MeTTa

- **'if'** takes three arguments: Condition, **then** Atom, **else** Atom.

```
(= (factorial $x)
  (if (> $x 0)
    (* $x (factorial (- $x 1)))
    1))
! (factorial 5) ; 120
```

- **'case'** takes two arguments: Variable, expression of patterns and their corresponding evaluations.

```
(= (factorial $x)
  (case $x
    ((0 1)
     ($_ (* $x (factorial (- $x 1)))))
  )
)
! (factorial 5) ; 120
```

Non-determinism

We know that MeTTa Atoms are immutable. What actually happens if we try to mutate a MeTTa Atom?

```
(= (bin) 0)
(= (bin) 1)
!(bin) ;[1,0]
```

```
; binary list
(= (gen-bin $n)
  (if (> $n 0)
      (:: (bin) (gen-bin (- $n 1)))
      ()))
!(gen-bin 2)
; [(| 0 (| 0 ())), (| 0 (| 1 ())), (| 1 (| 0 ())), (| 1 (| 1 ()))]
```

```
(= (factorial 0) 1)
(= (factorial $x)
  (* $x (factorial (- $x 1)) )
)
; This ends up in an infinite recursion. Why?
!(factorial 5)
```

Recursion Examples

```
; Find the length of a Cons list
(= (length ()) 0)
(= (length (Cons $x $xs))
    (+ 1 (length $xs)))
; List of length 3
! (length (Cons A (Cons B (Cons C ())))))
```

```
; Find the sum of all the multiples of 3 or 5 upto a certain number
(= (sum-multiple $num $sum)
    (if (> $num 0)
        (if (or (== 0 (% $num 3)) (== 0 (% $num 5)))
            (sum-multiple (- $num 1) (+ $sum $num))
            (sum-multiple (- $num 1) $sum))
        $sum)
    )
)
; Sum of multiples of 3 and 5 upto 10
!(sum-multiple 10 0)
```

Defining a List Data Structure in MeTTa

MeTTa lacks some basic data structures like lists. However, we can implement our own **List Data Structure** and **List Operations** in MeTTa.

```
(: List type)
(: Nil List)
(: Cons (-> Number List List))

!(get-type (Cons 1 Nil))      ; List
!(Cons 1 (Cons 2 (Cons 3 Nil))) ; [1,2,3]
```

```
(= (head $list)
  (case $list
    (
      (Nil Nil)
      ((Cons $x $xs) $x)
    )
  )
)
!(head (Cons 1 (Cons 2 (Cons 3 Nil))))
```

```
(= (head Nil) Nil)
(= (head (Cons $x $xs)) $x)

!(head (Cons 1 (Cons 2 (Cons 3 Nil))))
```

Hyperon Training Exercises

1. Create your own data structure for a binary tree in MeTTa.
2. Write the following list operations in MeTTa:
 - *length, is-member, append, max-value, min-value, push, pop, remove-element, remove-duplicate, map, filter, foldl, foldr, reverse, sort*
3. How can OpenCog Hyperon be the framework for Artificial General Intelligence?